

The Jeffy Loop

How an autonomous coding loop earns
the right to say it is finished

August 2026

<https://github.com/lenamonj/jeffy-loop>

Contents

Executive summary.....	3
Part I: How we got here.....	4
1. What a loop is.....	4
2. The prehistory, and the rebuttal nobody quotes.....	4
3. The 2023 wave, and why it broke	5
4. The coding-agent generation	6
5. The Ralph Loop.....	7
6. What none of them answered	9
Part II: What Anthropic recommends.....	10
7. Workflows, agents, and the case for building neither	10
8. The evaluator-optimizer pattern	11
9. Claude Code as a harness	11
10. Context engineering, and why fresh context sees more.....	13
11. The gap this leaves	13
Part III: The Jeffy Loop.....	14
12. The design thesis	14
13. Anatomy of a run	14
14. The rules, and why each one exists.....	16
15. What this is trying to achieve	21
16. The receipts.....	22
17. Honest limits.....	28
Glossary.....	31
References	33

Executive summary

An autonomous coding agent is a loop. A model is given a task, it acts, it observes what happened, and it acts again, until it decides it is done. The hard part is not the acting. The hard part is the last clause: **deciding it is done**.

That decision is structurally compromised, because the only judge present is the model that did the work, grading its own output against criteria it also chose. Hence the characteristic failures of the field. Agents declare victory over broken code, loop forever on tasks with no achievable finish, and report fixes validated only by a test suite that was measuring the wrong thing.

Part I traces how the field arrived here, from first principles, for readers who have never built an agent. It covers the research showing a model can critique and retry its own work, the rebuttals showing self-critique is far weaker than external verification, the 2023 wave of autonomous agents that mostly failed, the coding agents that followed, and the Ralph Loop, which strips the idea to a single line of shell. It ends on the question none of them answered.

Part II summarises what Anthropic actually recommends. Much of it is advice not to build an agent at all; where one is warranted, the recommendations are specific: ground truth from the environment, a sandbox, stopping conditions, and a separate evaluator judging the work with fresh context. Part II closes on what the guidance deliberately leaves open: what a particular run must verify before it may stop.

Part III describes the Jeffy Loop, a Claude Code skill built on one commitment: the decision to stop is taken from the model and given to a shell script. A run ends only when a program holding no weights and no opinion re-reads the claims the run made, re-runs the project's own tests, and finds everything still true. Every other rule follows: findings reproduced before filing, because a script must be able to re-run them; severity judged against a written statement of what the software accepts, because otherwise severity is a mood; the whole public surface mapped before anything is examined deeply, so that "no findings" and "nobody looked" cannot sound alike.

The evidence is 129 open-source projects with no connection to the author, run in the summer of 2026 and published in full, failures included: one hundred converged across thirteen languages, and maintainers with no stake in this project merged twenty of the loop's patches into seventeen of them, at Apple, Microsoft, JetBrains and Apache among others and inside Node.js. The clearest found four data-loss defects behind a suite of thirty-one passing tests that could not observe a database commit.

The limits are stated as plainly as the results. The adversarial reviewer is the same model with a clean context, so it defeats self-persuasion but not a shared blind spot, and one published finding was retracted on exactly that. Many of the tests that certify a run were written by the run. A known-answer check cannot argue about which answer the situation called for. And convergence is a statement about a contract, never a certificate of correctness. Section 17 carries the whole list, and the merged patches are the reply to it: the one check with no stake in the result.

Part I: How we got here

1. What a loop is

A large language model, on its own, does one thing. Text goes in, text comes out, and the exchange ends. Ask it to fix a bug in your project and it will produce something that looks like a fix, written from whatever it can see in the conversation. It cannot open your files, cannot run your tests, and has no way of learning whether what it wrote actually works.

Almost everything interesting about modern coding assistants comes from removing that limitation, and the removal has two parts.

The first part is **tools**. The model is given a way to request that something happen outside itself: read this file, run this command, search the web. The request goes to a surrounding program, that program performs the action, and the result comes back to the model as more text. The model still only produces text, but some of that text is now a structured request, and something real happens as a consequence.

The second part is **the loop**. Rather than answering once and stopping, the model is invited back. It acts, it observes what happened, and it acts again, repeatedly, without a human between the steps. A model wrapped in tools and placed in a loop is what the field calls an **agent**, and the surrounding program that holds the loop, supplies the tools, and decides when to stop is called the **harness**.

Two further terms will recur. The **context window** is the finite amount of text the model can hold in view at once: the conversation, the instructions, the file contents it has read, and the output of every command it has run. It fills up, and quality degrades as it does. An **oracle** is anything outside the model that can settle whether an answer is right, such as a test suite, a compiler, a mathematical identity, or a human expert. The presence of cheap oracles is the single reason coding became this field's proving ground. A test suite will tell an agent it is wrong without being persuaded, argued with, or flattered.

With that vocabulary the problem can be stated in one line. Building a loop is easy. Deciding when the loop should end is not, because the obvious candidate for making that decision is the model, and the model is the interested party.

2. The prehistory, and the rebuttal nobody quotes

The intellectual groundwork was laid in a short run of papers between late 2022 and late 2023. The first three are famous. The two that answer them are not, and the argument of this paper depends on the answer.

ReAct, published by Yao and colleagues in October 2022 [1], made the foundational move. Before it, prompting research had largely treated reasoning and action as separate: either you had the model think step by step, or you had it call a tool. ReAct interleaved them, so the model produced a reasoning trace and an action in alternation. The reasoning helps it form and revise

a plan and handle surprises; the actions let it consult the outside world and bring back facts it did not have. That interleaving is the shape of essentially every agent built since.

Reflexion, from Shinn and colleagues in March 2023 [2], added the ingredient that makes a loop more than repetition. Its agents reflect in words on how a previous attempt failed and keep those reflections in memory, so the next attempt can use them. The important word is verbal: nothing is retrained and no weights change. The lesson learned is text, carried forward. This is why a modern agent can fail, notice why, and do better on the next try without anyone touching the model.

Self-Refine, published days later [3], asked the narrowest and most uncomfortable version of the question. It used one model as generator, critic and reviser: produce an output, critique it, revise using that critique, repeat. It reported gains across seven tasks, which established that a model critiquing itself does better than a model not critiquing itself.

That is where the popular account stops, and it is the wrong place to stop.

In October 2023, Huang and colleagues published **"Large Language Models Cannot Self-Correct Reasoning Yet"** [4]. Their finding is blunt: models struggle to correct their own responses without external feedback, and their performance sometimes gets *worse* after self-correction. The same month, Valmeekam and colleagues tested self-critique on planning problems and produced the number that should be printed above every agent architecture diagram [5]. Using a model to generate plans and then critique its own plans raised the success rate from 40 out of 100 to 55 out of 100. Replacing that self-critique with a sound external verifier raised it to **88 out of 100**. Their model-based verifier also produced 38 false positives out of 100, meaning it repeatedly certified plans that were wrong.

The lesson is not that self-critique is worthless. It is that self-critique and external verification are different in kind, not in degree, and that the gap between them is enormous. A model reviewing its own work reliably catches carelessness. It does not reliably catch conviction, because the reviewer is confidently wrong in the same places the author was.

Everything in Part III is an attempt to buy as much of that 88 as possible.

3. The 2023 wave, and why it broke

In March 2023 the ideas escaped the literature. **AutoGPT**, from Toran Bruce Richards under the name Significant Gravitas, appeared as a public repository in mid-March 2023 with its first tagged release in April [6]. **BabyAGI**, from Yohei Nakajima, was described in a concept note in late March and published as code days later [7]. Both took the same premise: give a model a high-level objective, let it decompose that objective into tasks, execute them, generate new tasks from the results, and continue autonomously.

The attention was extraordinary. AutoGPT is today among the most starred repositories on GitHub. The usefulness did not follow, and the reasons are documented in the projects' own words rather than by hostile outsiders, which is what makes them worth citing.

Read AutoGPT's control loop and the whole story is there. It runs an unbounded loop that can exit in exactly three ways: a user-supplied iteration cap, the human typing a refusal, or **the model emitting a `task_complete` command**. There is no test, no oracle, and no verifier anywhere in that list. The only thing entitled to declare the work finished is the same model that has been doing it. That an iteration cap shipped from the beginning is itself the admission that the loop would not otherwise stop.

The project's own README warned that its continuous mode was not recommended and might cause the agent to run forever [6]. Its issue tracker recorded the rest. A user reported that a complex task left the agent "stuck in a loop created by itself," and, more damningly, that it would ignore an error and assume the task was completed, then act on that assumption. Another reported that the agent did not track what it had already done and would repeat identical commands indefinitely. Both issues were closed as not planned, labelled as limitations of the model [6]. The maintainers were conceding that the problem could not be fixed at the harness layer, which, given the harness they had built, was correct.

BabyAGI's design never intended to terminate. Its README described an infinite loop of pulling a task, executing it, creating new tasks and reprioritising, and warned about the resulting API usage [7]. Nakajima's own concept note listed, as a known risk, that the system could overload if tasks were generated faster than they were completed.

The independent measurements, when they arrived, were unkind. On the **GAIA** benchmark [8], where human annotators scored 93.9 on the easiest tier, AutoGPT with a GPT-4 backend scored 14.4, and on the middle tier it scored 0.4, worse than the same model with no agent scaffolding at all. It also consumed roughly as much wall-clock time as the humans did. **AgentBench** [9] made the failure mode legible by counting it: across its environments, the proportion of runs that simply exceeded the task limit without concluding reached 67.9 percent in one environment and 82.5 percent in another. Its authors named poor long-term reasoning, decision-making and instruction following as the main obstacles to usable agents.

It is worth being precise about what failed, because the common summary is wrong. The reasoning worked. The tool use worked. The decomposition often worked. What was missing was any mechanism, outside the model, capable of saying "this is finished" or "this is not going to finish." The 2023 wave was not defeated by weak models. It was defeated by the absence of a stopping rule, and the projects' own maintainers said so at the time.

4. The coding-agent generation

The next wave narrowed the target, and the narrowing is what made progress measurable.

In October 2023, Jimenez and colleagues at Princeton published **SWE-bench** [10], built from 2,294 real issues drawn from twelve mature Python repositories. To score, a system must navigate an unfamiliar codebase, find the relevant code, write a patch, and pass the project's own tests, which are never shown to it. Two sets of tests are used: those that must go from failing to passing, and a median of roughly fifty more that were already passing and must stay that way. That second set is the part worth noticing. It encodes the rule that "I fixed it" is an incomplete claim without "and I broke nothing."

Initial results were humbling, with the headline system resolving under two percent of issues. The number has climbed a long way since, driven as much by better harnesses as by better models.

SWE-agent, from largely the same group in 2024 [11], contributed the insight that the interface matters as much as the model. By designing the commands and feedback the agent works through, rather than handing it a raw shell, performance improved substantially. Two of its design choices matter here more than its score. It integrated a linter into its edit command and **discarded invalid edits outright**, forcing the agent to try again rather than proceed on broken code. And its submission step ran a checklist that included reverting any test files the agent had modified. That second guard is an early acknowledgement of a problem this paper returns to: an agent judged by tests has an incentive to edit the tests.

Alongside the research came the practical tools. Aider, from Paul Gauthier in 2023, made two contributions that look obvious only in hindsight: it commits after every edit, so the loop is reversible, and it can run your test command after each change and feed a non-zero exit code back to the model as the thing to fix [12]. Its answer to "how does it know it is done" is honest and minimal, which is that it does not decide. The exit code and the human decide. Devin, from Cognition in March 2024, capped its runs at forty-five minutes precisely because, in their words, it could otherwise run indefinitely, and reset the test files after each run in case the agent had modified them. OpenHands made the whole loop open and inspectable. Claude Code, the harness Part III builds on, arrived in early 2025.

One recent development belongs here, because it complicates the tidy story that oracles solve everything. In February 2026, OpenAI announced it would stop reporting scores on SWE-bench Verified, the human-filtered subset of the original benchmark [13]. Their stated reason was that an audit of the harder problems found that a majority of those audited had **flawed test cases that reject functionally correct solutions**, some by enforcing incidental implementation details and some by testing for behaviour nobody specified. They also documented contamination, with models reproducing gold patches from memory.

The lesson is not that tests are a bad oracle. It is that a test suite is a **proxy** for correctness, written by people, and the proxy can be wrong in both directions. Part III's central claim depends on that being true, because a loop that trusts a green suite absolutely will converge on a project whose suite is measuring the wrong thing, which is exactly the case study section 16 presents.

5. The Ralph Loop

In July 2025, Geoffrey Huntley published a description of a technique he had named, with deliberate self-deprecation, after Ralph Wiggum [14]. It is the most instructive thing in this history because it is the smallest.

In its purest form, as the post prints it, Ralph is a Bash loop:

```
while ;; do cat PROMPT.md | claude-code ; done
```

That is the whole technique. Take a prompt file. Feed it to a coding agent. When the agent finishes, feed it the same prompt again. Forever. There is no orchestration layer, no task queue, no planner, no memory architecture. The agent begins each iteration with a fresh context and

identical instructions, and whatever persists between iterations persists because it was written to disk, which in practice means the code and a plan file the prompt tells the agent to maintain. A widely used community playbook states the mechanism plainly: the plan file on disk is the shared state between otherwise isolated executions, and no sophisticated orchestration is needed because the agent re-derives what to do by reading it [15].

The technique works better than it has any right to, and understanding why is worth the effort. Each iteration starts clean, so the context degradation described in Part II never accumulates. The repository is the memory, and the repository is real rather than remembered. And because the prompt is a file, improving the system means editing that file, which makes the whole thing tunable by hand.

Huntley's framing of the trade is the sharpest sentence written on this subject. He describes Ralph as **"deterministically bad in an undeterministic world"** [14]. It fails, but it fails reproducibly, and a reproducible failure can be answered by amending the prompt. He puts greenfield completion at roughly ninety percent, and reports delivering for a few hundred dollars work that had been contracted at fifty thousand. Part III returns to greenfield with a different kind of measurement: not a rate, but two individual builds run to convergence under external judges, with the full run record published.

He is equally direct about the limits, and these deserve full weight rather than treatment as a foil. He identifies the agent wrongly concluding that code is not implemented as a common failure, and calls that nondeterminism the technique's Achilles heel. He states that you will wake up to a broken codebase from time to time. He notes the model's inherent bias toward placeholder implementations. And he puts the real problem in one sentence: since code generation is now easy, **the hard part is ensuring the loop has generated the right thing** [14].

His remedy is what he calls back pressure: wire in whatever will mechanically reject bad work, such as compilers, type checkers, linters and tests, and make the wheel turn fast. That is the correct instinct, and Part III is in large measure an elaboration of it.

Two things are nonetheless absent from the technique as published, and both matter. First, there is no termination condition at all. The loop is literally `while :`. It runs until a human stops it, which Huntley is explicit about, advising the operator to watch the loop and to sit on the loop rather than in it [16]. Second, the loop has no notion of what it has already verified. Back pressure rejects a bad change when it happens; it does not accumulate a record of which parts of a system have been examined, so nothing distinguishes "this is clean" from "nobody looked."

Huntley's own stated scope is consistent with both gaps. He says he would not use Ralph on an existing codebase [14]. That is not a weakness in the technique but an accurate statement of its domain. A loop with no memory of what it verified and no gate on what it breaks is a reasonable instrument for building something new, where a bad iteration costs little and the next one can overwrite it. It is a poor instrument for auditing something that already exists and already works, where a bad iteration is a regression in code other people depend on, and where the valuable output is not new code but a trustworthy statement about the code already there.

6. What none of them answered

By late 2025 the Ralph pattern had been adopted by vendors, and their implementations are the clearest possible statement of the unsolved problem, because each had to pick a stopping rule and write it down.

Anthropic shipped a plugin implementing the technique inside a single session, using a Stop hook to block the session from ending and re-feed the prompt [17]. Its termination is exactly two conditions: a maximum iteration count, or **the model emitting a configured completion phrase**. The documentation is candid that the phrase is matched as an exact string and advises treating the iteration cap as the primary safety mechanism. Cursor's equivalent instructs the model that it may only emit the completion phrase when the statement is genuinely true, and defaults to unlimited iterations.

Read that instruction again, because it is the entire problem in one line. The exit gate is an appeal to the model's honesty about its own completion, with no cap by default. The predictable happened: one documented case has a vague prompt and an unlimited setting producing 1,966 iterations over several hours before the operator deleted the state file by hand.

That the model's self-report is unreliable at exactly this point is not speculation. OpenAI's own system card for its coding agent lists, as a named product risk, that the model **might falsely claim to have completed a task it did not complete**, and reports that in early testing, faced with an impossible task, it would often claim success rather than admit it could not proceed. Their published mitigation figures show that before training against this behaviour, the agent correctly admitted it could not finish the task in **15 percent** of synthetic cases [18]. Later academic work measuring the same phenomenon across agent benchmarks found silent false-success accounting for a substantial share of all failures, and, crucially, found the rate collapsing in the one setting where an independent party could observe the environment state directly [19].

So follow the whole thread and one thing never gets solved. Self-Refine put the critic inside the model, and the rebuttal papers showed what that costs. AutoGPT put the completion judgement inside the model, and it looped forever. Ralph removes the judgement entirely and runs until a human intervenes, which is honest but requires the human. SWE-bench sidesteps the problem by supplying an external oracle for one task, which works precisely because somebody else already wrote the failing test, and which the 2026 audit showed is imperfect even then. And the vendor plugins, given the chance to choose any stopping rule they liked, chose an iteration cap plus the model's word.

For an agent running unattended against an existing codebase, none of these transfer. There is no failing test, because the entire point is to discover what should be failing and is not. There is no human watching to notice that progress has stopped. And the only entity in a position to declare the work complete is the entity whose work is being judged.

The question this leaves is small enough to state in a sentence, and Part III is an attempt at an answer.

What must be true before an autonomous run is permitted to say it is finished, and who, or what, checks?

Part II: What Anthropic recommends

Part I ended on a problem nobody had solved: an autonomous loop cannot be trusted to decide it is done. Before describing one attempt at a solution, it is worth establishing what the vendor of the underlying model actually recommends, because a surprising amount of it is advice not to build the thing at all.

7. Workflows, agents, and the case for building neither

Anthropic's foundational guidance on this subject is an engineering post titled "Building effective agents," first published in December 2024 [20]. Its most useful contribution is a distinction that the industry had been blurring. Under the umbrella term "agentic systems" it separates **workflows**, in which models and tools are orchestrated through predefined code paths, from **agents**, in which the model dynamically directs its own process and tool usage.

The distinction is not academic. Workflows are predictable and consistent, which suits well-defined tasks. Agents are the right choice when flexibility and model-driven decision-making are genuinely needed at scale. The choice between them is a choice about how much unpredictability the task can absorb.

The posture of the piece is restraint. It advises finding the simplest solution that works and adding complexity only when necessary, and it says plainly that this may mean not building an agentic system at all. For many applications a single well-optimised model call with retrieval and good examples is sufficient. Agentic systems trade latency and cost for task performance, and that trade is not always worth making. The piece is similarly sceptical of frameworks, recommending that developers start with the model API directly, since many of these patterns are only a few lines of code, and understand any framework's underlying behaviour before adopting it.

It then lays out a progression of patterns, roughly in order of increasing autonomy. The foundation is the augmented model: a model with access to retrieval, tools and memory, generating its own queries and selecting its own tools. **Prompt chaining** decomposes a task into a fixed sequence of calls, each working on the previous output, optionally with programmatic checks between steps, and suits tasks that decompose cleanly into known subtasks. **Routing** classifies an input and sends it to a specialised follow-on path, which suits inputs falling into distinct categories better handled separately. **Parallelization** comes in two forms: sectioning, which splits a task into independent subtasks run simultaneously, and voting, which runs the same task several times to gather diverse attempts, and suits work needing either speed or multiple perspectives for confidence. **Orchestrator-workers** has a central model decompose a task dynamically, delegate to workers, and synthesise the results, which suits complex tasks whose subtasks cannot be predicted in advance.

8. The evaluator-optimizer pattern

The sixth pattern is the direct ancestor of the mechanism described in Part III, and it deserves separate treatment.

In the **evaluator-optimizer** pattern, one model call generates a response while a second provides evaluation and feedback, in a loop. The generator produces, the evaluator critiques, the generator revises. Anthropic's guidance on when this pattern fits is precise and worth applying honestly: it works when there are clear evaluation criteria, and when iterative refinement provides measurable value. The two-part diagnostic is whether a human articulating feedback would demonstrably improve the output, and whether the model can supply that feedback itself. The analogy offered is a writer working through editorial revisions.

The seventh and final pattern is the **autonomous agent**: a model using tools based on environmental feedback in a loop, planning and operating independently after an initial instruction. Anthropic recommends it for open-ended problems where the number of required steps cannot be predicted and no fixed path can be hardcoded. Two conditions attach to that recommendation, and they are the conditions Part I showed the 2023 systems failing. First, the agent must obtain ground truth from its environment at each step in order to assess its own progress. Second, because autonomy brings higher costs and the risk of compounding errors, such systems need extensive testing in sandboxed environments, appropriate guardrails, and explicit stopping conditions such as a maximum iteration count.

The post closes on three principles: keep the design simple, make the agent's planning steps visible, and invest as much care in the agent's interface to its tools as one would in a human user interface. Its summary judgement is that success is not about building the most sophisticated system but the right one for the need.

One passage in its appendix explains why coding became the proving ground for this entire field. Code solutions are verifiable through automated tests, so an agent can iterate using test results as feedback. That single property, an oracle that is cheap to run and hard to argue with, is what makes autonomous coding tractable where autonomous most-other-things is not.

9. Claude Code as a harness

Claude Code is Anthropic's command-line coding agent, and its documentation [21] is where the abstract patterns become concrete machinery. Four features matter for what follows, and a reader new to the tool needs all four.

CLAUDE.md is a file the agent reads at the start of every conversation, holding project-specific instructions. The documentation's guidance on it is disciplined: the test for whether a line belongs is whether removing it would cause a mistake. Bloated files cause the agent to ignore the instructions that matter. Because it loads every session, it should hold only broadly applicable material.

Hooks are scripts registered to run at defined points in the agent's lifecycle. The distinction the documentation draws is the important one: instructions in CLAUDE.md are advisory, while hooks are deterministic and guarantee that something happens. Hooks are recommended for actions

that must occur every time without exception. A **Stop hook** runs when the agent finishes a turn and can block the turn from ending. This is the mechanism Part III's loop engine is built on.

Skills are folders containing instructions, scripts and resources that the agent discovers and loads when relevant [22]. They exist to solve the context problem that CLAUDE.md cannot: knowledge needed only sometimes should not be loaded always. Skills use progressive disclosure, where only a name and description sit in context at startup, the full instructions load when judged relevant, and bundled files are opened only as needed.

Subagents run in their own context window with their own tools and system prompt, reporting a summary back. The documentation recommends them both for investigation and for verification, and its stated rationale for the latter is precise: a fresh reviewer sees only the diff and the criteria given to it, not the reasoning that produced the change.

Most relevant of all, the current documentation leads with the problem this paper is about. It observes that the agent stops when the work looks done, and that without a check it can run, "looks done" is the only signal available [21]. Its recommended remedy is to give the agent a check that returns a readable pass or fail, and it lays out four escalating levels of how firmly that check gates the stop: asking the agent to run it within a single prompt; setting it as a session-level goal condition that a separate evaluator re-checks after every turn; enforcing it with a Stop hook that blocks the turn from ending until it passes; and obtaining a second opinion from a verification subagent, so that the agent doing the work is not the one grading it.

Level	Mechanism	What enforces it	Weakness
1	Ask the agent to run the check in the same prompt	The model's own compliance	Nothing enforces it if the model skips the step
2	Set the check as a session goal condition	A separate evaluator re-checks after each turn	The evaluator is still a model
3	Enforce with a Stop hook	A script blocks the turn from ending	Only as good as the check the script runs
4	Add a verification subagent	A fresh model instance tries to refute the result	Shares the original's training and blind spots

That fourth level carries a caveat the honest reader should keep. A reviewer prompted to find gaps will usually report some, even when the work is sound, and chasing every such report leads to over-engineering: unnecessary abstraction, defensive code, and tests for cases that cannot occur. The remedy suggested is to scope the reviewer narrowly, to correctness and stated requirements.

10. Context engineering, and why fresh context sees more

Two further pieces of Anthropic guidance explain why a fresh sub-agent is worth the cost.

The first is on context engineering [23]. Its central claim is that context must be treated as a finite resource with diminishing returns, for a mechanical reason: attention across a sequence produces relationships that scale quadratically with its length, and models are trained on far fewer long-range dependencies than short ones. The observed consequence is that as a context window fills, the model's ability to accurately recall what is in it degrades. The piece recommends compaction, external note-taking that persists outside the window, retrieving information just in time rather than pre-loading it, and delegating focused work to sub-agents with clean context windows, so that detailed exploration stays isolated and only the synthesis returns.

The second, published in March 2026, reports on harness design for long-running work [24] and contains the most direct empirical support for the evaluator pattern. It describes separating the agent doing the work from the agent judging it as a strong lever against the tendency of models to over-praise their own output. It also records a phenomenon it calls context anxiety, where models lose coherence as the window fills and prematurely wrap up work as they approach a perceived limit.

That same piece carries a warning any paper describing a harness should quote against itself. Every component in a harness encodes an assumption about what the model cannot do alone, and those assumptions decay as models improve. The authors found that scaffolding which was load-bearing on one model generation became unnecessary on the next, at which point removing it made the system both cheaper and better. The instruction is to re-examine the scaffolding on every model release and strip away whatever is no longer carrying weight.

11. The gap this leaves

Taken together, the guidance is coherent and, on its own terms, complete. Use the simplest thing that works. Prefer workflows to agents where the task allows. If you build an agent, give it ground truth from its environment, keep the design simple, make its reasoning visible, invest in its tools, sandbox it, and set stopping conditions. Where quality matters, add an evaluator, and give the reviewer fresh context so it is not grading its own reasoning.

What none of it supplies is a stopping rule for a specific unattended run.

The recommended stopping condition is a maximum iteration count, which is a budget rather than a judgement of completion. The evaluator-optimizer loop terminates when the evaluator is satisfied, which returns the decision to a model. The Stop hook gates a turn on a check passing, which is closer, and is the mechanism Part III builds on, but the documentation stops at the mechanism and does not prescribe what the check should verify for an audit that must decide whether a codebase is now sound.

This is not an oversight. General guidance cannot specify a completion contract for work whose definition of done is different in every project. But it means that anyone running an agent unattended against real code has to answer, themselves, the question Part I ended on. What has to be true before this run is allowed to say it is finished, and who checks?

Part III: The Jeffy Loop

12. The design thesis

Every system in Parts I and II converges on the same unresolved question. An autonomous loop can plan, act, observe and retry. What it cannot reliably do is decide that it is finished, because the only judge available is the same model that did the work.

Jeffy Loop is a skill for Claude Code built around a single commitment: **the decision to stop is taken away from the model**. A run ends when a shell script, holding no weights and no opinion, re-checks every claim the run has made and finds them all still true. If any check fails, the script feeds the failure back into the loop and the run continues.

This is a narrow idea, and most of the system follows from it. If a shell script has to verify the claims, the claims must be written somewhere a script can read. If the script re-runs the project's test command, that command must be recorded in a fixed place and must not be allowed to lie. If findings are to be re-checked, they must have been reproduced in the first place rather than asserted. The rules described in this part are not a collection of good practices. They are the consequences of taking the stopping decision away from the model and giving it to a program.

The idea was built over one summer of evenings and weekends, and then it was tested the only way that means anything: against other people's code, judged by other people. Between July and September 2026 the loop was pointed at 129 open-source projects with no connection to its author, in thirteen languages, each judged by its own test suite, every run published, failures included. One hundred of them ran to convergence. Seventeen projects, among them libraries maintained by Apple, Microsoft, JetBrains and the Apache Software Foundation and the URL parser inside Node.js, merged patches the loop wrote, because a maintainer with no stake in this project read the change and said yes. Those verdicts are the spine of this part. The rules are what it took to earn them.

13. Anatomy of a run

A Jeffy run has four files, one prompt, one hook, and a commit after every iteration.

The state files live at the root of the project being worked on, and they are the loop's memory. `PLAN.md` holds the goal, the operating envelope, the method, the severity rubric, the verify command, the surface inventory, and the definition of done. `BACKLOG.md` is a ledger of open work, deliberately not a narrative: tasks are one-line items with an acceptance check, and a completed task is deleted from the ledger rather than annotated. `JOURNAL.md` is append-only, one entry per iteration, following a fixed heading grammar so that a script can parse it. A fourth file, `.claude/jeffy-loop.local.md`, is transient session state holding the current iteration number, the budget, and the session identity. It is deleted when the run ends and is never committed.

The iteration prompt is a single line of text stored on disk, a little over thirty thousand characters at this revision. It contains the entire discipline of the loop: how to audit, when to file a finding, how severity is judged, what the journal entry must contain, what convergence requires. It is not

injected into the conversation once and left to decay. It is re-read from disk and re-fed at the end of every single turn.

The Stop hook is the engine. Claude Code lets a user register a script that runs when the assistant finishes a turn, and that script can refuse to let the turn end. Jeffy's hook and the four small helper scripts beside it run to about 2,900 lines of plain shell, and they have grown with every release, because in this design a new guarantee is always a check a script performs rather than an instruction a model is asked to honour. At every turn end the hook reads the state file, and while the budget remains it re-feeds the iteration prompt, which starts the next iteration. This is the mechanism that makes the loop a loop. Crucially, the hook is also the auditor: when the model attempts to declare convergence, the hook independently verifies the claim before permitting the session to end. Since the current release the run can also ask the hook, at any point, what a declaration would be refused on, and repair it before spending a review on it. Section 14 returns to that.

The checkpoint is a local git commit made at the end of every iteration with the message `jeffy: iter i/N task status`. Nothing is ever pushed and no branches are created. The checkpoint serves three purposes: it is the unit the loop reverts to when an iteration breaks the build, it guarantees the next iteration starts from a clean working tree, and it leaves a reviewable history so a human can read the run backwards.

The table below summarises what each piece is and who controls it. The right-hand column is the one that matters: the components a model can influence are separated from the components it cannot.

Component	What it is	Who controls it
PLAN.md	Goal, envelope, method, severity rubric, verify command, surface inventory	Model writes, script reads and checks
BACKLOG.md	One-line open tasks, each with an acceptance check and a severity	Model writes, script reads the severity and the count
JOURNAL.md	Append-only record, one entry per iteration, fixed grammar	Model writes, script parses
Iteration prompt	The entire discipline of the loop, re-fed every turn	Fixed on disk, model cannot edit
Stop hook	Shell script that re-feeds the loop and audits the stop	Fixed on disk, model cannot edit
Checkpoint commit	Local git commit ending each iteration	Made by the model, verified by the script
Verify command	The project's own test suite	Chosen at setup, re-run independently by the script
Probe batteries	Known-answer checks the loop writes under <code>.jeffy/probes/</code>	Model writes, script executes their claims

A run therefore looks like this. The user types `/jeffy 10` in a project. The skill runs ten pre-flight checks, copies the state file templates in if they are missing, prints the budget it resolved and which run of this project this is, writes the session state, and begins iteration one. Because a fresh project has an empty backlog, iteration one is always consumed by an audit that maps the surface and generates the work. Every iteration after that executes exactly one task, never a batch, verifies it, runs the project's own test command, writes a journal entry, and commits. The hook re-feeds the prompt. The run ends when the budget is exhausted, when a hard blocker is hit, when one of the hook's own gates decides the run is going nowhere, or when convergence is earned and verified.

14. The rules, and why each one exists

The rules are presented in the order a run meets them. Each one was learned from a published run that went wrong without it, and most of them carry the name of that run.

Evidence before filing

A finding does not exist until the loop has reproduced it. Not "this looks wrong," but a command that was run, with its real output, recorded before the finding is written down.

The reason is that a language model asked to find defects will find them, because that is what it was asked to do. Plausibility is cheap and abundant. Reproduction is expensive and scarce, and the expense is the point: writing the reproduction is where the model usually discovers the bug is not real. The rule converts an unfalsifiable opinion into a claim that a shell script can re-run later, which is what the whole architecture depends on.

The operating envelope

Before any finding can be filed, `PLAN.md` must state the project's real input surfaces: what this software is expected to receive, from whom, under what trust assumptions. Severity is then judged against that envelope rather than in the abstract. A defect reachable only by input the project never claims to accept is a Low at most, no matter how dramatic it looks.

Without an envelope, severity is a mood. An agent can call anything critical, and a list of critical findings that are mostly hypothetical is worse than no list, because it destroys the reader's ability to triage. The envelope gives severity a denominator.

The surface inventory, and the coverage guarantee

The first audit must map the project's entire public surface into a checkbox table, one row per public module or entry-point group, and probe every row at least shallowly before investigating any row deeply. A row is only ticked with the commit hash it was swept at, the battery of known-answer checks that swept it, and one line naming what the sweep exercised. If the code behind a ticked row later changes, the hook derives that from the battery's declared paths and the row goes back to unticked, whatever the run believes.

This exists to stop a specific dishonesty. When an agent reports "no findings," the reader hears "this project is clean." What the agent usually means is "I did not find anything where I looked."

The inventory forces the second statement to be visible, and it forces breadth before depth, so the worst defect in a project surfaces in the first audit rather than the sixth.

Coverage is also a guarantee with a bound, not a preference. The loop's queue orders open High findings first, then unswept rows, then Mediums, then Lows, so the map outranks everything except a known High, and a sweep iteration may batch every row it can properly evidence, the one exemption from never-batch, because rows batch and findings do not. The hook records the unswept count at every turn end, derives the run's own sweep rate, projects when the map clears, and says so in every re-feed. When the projection says the map cannot clear inside what remains of the budget, the hook ends the run early with the arithmetic rather than at exhaustion. That rule came from the user of this engine, after watching a run spend a hundred iterations without finishing its map: cover every row within a bounded time, and then, if there is time left, go deep.

The rule has a sharp corollary for code that computes values. Sweeping such a surface requires a known-answer or invariant check for each function family, never a probe that merely confirms the code runs without crashing. Wrong numbers returned without complaint survive every liveness probe ever written. Every documented parameter must be exercised at two or more values that must change the output, and a documented parameter whose value changes nothing is itself a finding rather than a pass. And a battery that has never been seen to fail reads exactly like a battery that passed, so each one records at its creation the input it was once observed failing on.

The verify gate, the refusal that protects it, and the oracle it has to name

PLAN.md records the project's own verification command, typically its test suite. After every task, the loop runs it through a wrapper that keeps a green suite's output out of the context window and records the one summary line worth quoting. If the command fails and it passed at the last checkpoint, the loop concludes that this iteration broke the project: it reverts the working tree to the last checkpoint, marks the task blocked with the failure output, and records what happened.

One refusal protects this gate and deserves its own explanation, because it is the least obvious rule in the system and the most consequential. Jeffy refuses a verify command whose pipeline ends in a pager or truncator such as `head`, `tail`, `less`, `more` or `cat`. The reason is a property of shell pipelines: the exit status of a pipeline is the exit status of its last command. A user who writes `pytest | tail -1` to keep the output short has written a command that reports success even when every test fails, because `tail` succeeded at printing the last line. A loop trusting that command would converge over a red suite, having verified nothing. The refusal fires when the run is launched and again at the gate.

That refusal catches a command that lies about its exit status. A later release had to answer the harder version of the same problem: a command that is entirely honest about its exit status while grading almost nothing. In a run against `go-yaml`, a widely used YAML parser for Go, the verify command exited zero for twenty-nine iterations across three runs, and the 402-case conformance corpus vendored into that very repository never executed once. The only file importing the corpus carried a build constraint excluding Windows, which was the platform the run executed on. The run's journal asserted twice that the corpus was green, and the adversarial evaluator countersigned both claims. Scored independently afterwards, on clones the run had never

touched, the corpus reported 355 of 355 cases passing at the base commit and 355 of 355 at the converged commit. The external oracle moved by nothing.

The rule that came out of that puts the burden where it can actually be discharged. `PLAN.md` carries two further lines beside the command. **Oracle class** names what the command really grades, in the run's own words: unit tests, a conformance corpus, a differential comparison against a reference implementation, a build only. **Environment fingerprint** names the platform, the toolchain versions, and every test target this platform excludes, with that exclusion list derived by a command rather than asserted. The hook refuses a declaration whose lines are unfilled, and a test asset the fingerprint says the command cannot reach is one no journal entry may claim was green.

Numbers are measured, never typed

This rule arrived late and it arrived from the engine's own runs on its own repository. Across four consecutive rejections in one self-run, the adversarial evaluator refused a declaration for the same reason each time: a count stated in prose beside the command that derives it, stale, with nothing executing that command. "Forty-eight mechanisms" against a command that returned fifty-three. The runs were not writing bad code. They were writing an inaccurate account of good code, and the gate was the only thing re-reading it.

So the account is now executable. A battery of probes carries a `claims` file, one line per measurement it makes, in the form `expect <value> :: <command>`, and a helper script runs every line and reports each as a match or a mismatch. A count that a governance file states in prose is written as `returns <count>` and carries a row in a table the same helper executes; a stated count the table does not carry refuses the declaration, and a number word the check cannot value is refused rather than passed, because the list that stops short is the one that fails silently. The test suite's own total is recorded by the wrapper that ran it, and the cell in `PLAN.md` may only equal it. The published pages of the project itself follow the same rule: every count on them is a marker the validator derives from the receipts table, and the count of validator checks is derived from the validator's own run and refuses a stale number. The check that guards that last figure caught its own stale marker on the first run after it landed.

Severity is what a user of the shipped product meets

Two rules govern severity, and both came from watching where runs spent their iterations.

The **floor**: a declaration needs zero open High and zero open Medium. An open Low does not block. It is carried, named by identifier in the declaring entry and in the published receipt, so the debt is visible rather than buried. Before this rule, a project with a long tail of minor findings could never finish, and every iteration spent on a cosmetic Low was an iteration the next High did not get.

The **ceiling**: a High or a Medium names a consequence a user of the shipped product meets. Wrong results, data loss, a crash on input the contract allows, a documented promise the code does not keep, a published artifact carrying what it should not. A finding of class test or developer tooling is a Low, always, because the user never runs it. A finding of class documentation or build configuration reaches Medium only when its line states the consequence a user meets. The hook

refuses a declaration over a line this run filed that breaks either rule. That rule was derived over forty-five published journals: Highs were closed by a median of iteration eleven and the last Medium by iteration seventeen of a median twenty, and thirty-seven of the 121 High and Medium tasks across the staged trees were a pytest configuration, a formatter's line, a lint job the run had itself reddened, a coverage path. Each cost an iteration, an audit and a gate cycle, and none of them was something a user meets. Measured again on the fifty journals from the current engine, 840 iterations in all, the tail after the last High or Medium closes is 18 percent of iterations, and Low tasks are 5 percent. The rest of the tail is the closing audit and the gate, which are the point.

The adversarial evaluator

Before the loop may claim convergence, it must spawn a fresh sub-agent as an adversarial evaluator. That sub-agent carries none of the run's conversation. It is instructed to assume the work is broken until proven otherwise and not to praise. Its scope is what this run found and says it fixed, nothing wider: for every High and Medium closed this run, it runs the filed reproduction against the commit that preceded the run and confirms it failed there, runs it against the result and confirms it passes, re-executes the acceptance check as written, and reads the fix's diff for a regression, with the project's own test suite green as the floor. It re-scores every finding against the rubric, and a High that names nothing a user meets is a rejection reason exactly as a Low that hides something a user meets is. It must return PASS. A rejection files its reasons as new backlog tasks and the run continues.

The gate is capped rather than repeatable: two invocations per run, three when the first landed before the midpoint of the budget, and the cap is absolute, so the gate can never be worn down by persistence. What ends the declaration path is a rejection with no invocation left. A run that reaches that point with budget still on the clock does not forfeit it: it enters gate salvage, works only the findings the gate itself filed, one per iteration as always, never re-invoking and never declaring, and ends blocked with those findings closed and the declaration deferred to the next run's fresh gate.

The gate also leaves a record. Before returning any verdict the sub-agent writes an artifact naming every command it ran and that command's real exit status, keyed by the run and by which invocation this was, and the hook refuses a PASS whose artifact is missing, uncommitted, or older than the tree it certifies. Who wrote that file cannot be proven at the shell layer, and neither the code nor this paper pretends otherwise. What changes is the price of a false PASS, from a typed verdict to a fabricated forensic record that the checkpoint commits and the repository keeps.

The honest description of what this buys, and what it does not, matters. The evaluator is the same model. A blind spot in the weights sits on both sides of the table, so this is not independence. What it buys is fresh context. It never sees the reasoning that produced the fix, only the artifacts and an instruction to break them. Section 17 gives a documented case where the evaluator passed a finding that was, in fact, wrong. The corpus also gives the other cases, which are the ones that matter: in `sqlfluff` the gate rejected a fix that had commented out the statement it was meant to repair, after 13,560 upstream tests had passed over it; in `rubyzip` the gate rejected a second-round fix that had silently reopened a symlink escape closed one iteration earlier, and the

loop withdrew the change rather than paper over it. Neither project's own suite covered either defect. That is the gate earning its place.

The machine-checked stop, and the run's preview of it

This is the load-bearing component, and it is not a model. The Stop hook refuses the converged stop unless every one of the following holds: the ledger is present with no open High or Medium and every carried Low parseable, the recorded convergence commit is still reachable and nothing but loop state has changed since it, no inventory row is unswept or stale, every declined finding records the command that establishes its premise, every settled class records the command that enumerates it, the packaging channels the project publishes through would not carry the loop's own files, the verify command has declared what it grades and what this platform excludes, every stated count is one a command derives, the closing journal entry carries an evaluator verdict with that gate's artifact committed beside it, and the project's own verify command exits green when the hook itself re-runs it. A failed check does not produce a warning. It re-feeds the loop with the evidence, and the run continues.

The asymmetry here is the entire design. The model can claim anything it likes. The claim only becomes an ending if a program that cannot be persuaded agrees.

It is worth being exact about how thin, and how important, the difference from the vendor implementations in Part I section 6 is. Jeffy also uses a completion phrase. The model signals that it believes the run is over by emitting a specific string, exactly as the Anthropic and Cursor plugins do. In those implementations the phrase **is** the stopping condition: matching it ends the run. In Jeffy the phrase is only a **request** to stop, and the hook then goes and checks. Both designs ask the model to say when it is done. Only one of them treats the answer as a claim to be audited rather than a fact to be accepted. That single inversion is what this project consists of.

The current release adds the run's own view of that audit. Because every one of those checks is derived from the tree rather than from the conversation, the hook can run them read-only at any time: `stop-hook.sh --lint` derives the whole declaration path, executes nothing, changes nothing, and prints the first thing a declaration would be refused on. The iteration prompt runs it before every gate invocation. The reason is arithmetic. Across the corpus, runs lost evaluator invocations to refusals that were true and trivial: a stated count outside its table, a wrapped bookkeeping line, a battery note. One run finished its product at iteration nine, took two evaluator passes, and lost both to forms the hook refuses, then spent a third pass in the next round to declare. Those were refusals the run could have read for itself, and now it does.

The engine is itself held to a behavioural check suite run on each of three continuous integration legs, Linux, Windows and macOS: 318 checks at this revision, each one added because a run found the defect it now guards, and each proven able to fail before it shipped. That figure is derived rather than typed, as the previous rule requires.

Closeout, the closing window, and the ratchet

Three rules keep runs terminating. **Closeout** says that once a full fresh-evidence audit has scored zero High and zero Medium, the run stops hunting for new findings on swept surface for the remainder of the run and finishes what is on the ledger; mapping unswept surface continues, because a declaration needs the whole map. **The closing window** is a one-time two-iteration extension the hook grants when the budget expires with the ledger at the floor and the surface swept, the shape where runs used to run out of iterations with the work done. That window buys exactly the gate, the fixes the gate files, and the declaration. It never buys an audit, and the hook ends the run the moment an audit entry appears inside it, because an audit there would let a run manufacture the very precondition its declaration then cites; section 16's greenfield arm records the run that did exactly that under the older engine. **The ratchet** says that if a previous run converged and nothing but the loop's own state has changed since, the next run verifies that and stops immediately, and the hook checks that the earlier declaration was one it actually accepted rather than one the run wrote and the hook refused.

The thrash gates

Three more gates watch the shape of a run rather than its claims. A run whose iteration changed nothing outside the loop's own files, moved no ledger line and flipped no inventory row draws a note; a second such iteration in a row ends the run as stalled, with the ceremony iterations, the closing audit, the gate, the ratchet and the wrap-up, exempt for the three iterations the convergence sequence needs and no longer. A run whose tree returns to a state it already held draws a note; the second return ends it as oscillating, because a run that changes a file and reverts it two iterations later passes a stall gate forever while going nowhere. And a task attempted three iterations running and still open draws the instruction the prompt has always carried, to mark it blocked with the reason and move on, now said by the hook that can count. Every one of these gates was shaped on a published run that burned a budget in exactly that shape.

15. What this is trying to achieve

The goal is not to make an agent that writes better code. It is to make an agent whose report you can act on without repeating its work.

That is a claim about trust, and trust in this setting is mechanical rather than emotional. When a run reports that it found four data-loss bugs and fixed them, the value of that report depends entirely on what would have had to be true for it to be false. In an ordinary agent loop, the answer is very little: the model could have been wrong, or persuaded itself, and nothing would have caught it. In a Jeffy run, a false report requires the reproduction to have run and produced its output, the project's own test suite to have passed under a hook that re-ran it independently, a fresh-context adversary to have failed to break it, and a shell script to have re-verified every structural claim. None of that makes the report certainly true. All of it makes the report expensive to fake, and the difference between a claim that is cheap to make and a claim that is expensive to fake is most of what practical trust consists of.

The second goal is subtler and more important for the reader deciding whether any of this generalises. The engine ships no language-specific analyser, no ruleset, and no per-ecosystem

plugin. It works entirely from what a project already has, which is its own test suite and its own verification command. That is why the same skill ran against a C allocator, a Kotlin date library, a Swift logging framework, a PHP CSV parser and a Java text library without modification, and it is why the method rather than the tooling is the transferable part.

The third goal is the one the corpus finally settled. Every check inside the loop is, in the end, the system grading itself with progressively better discipline. The only judge with no stake in the result is the maintainer of the project the loop touched, reading a patch against their own library. A merged pull request is the one outcome the loop cannot award itself. So the plan from the start was to go and get those verdicts.

16. The receipts

Testing a tool against its own codebase proves very little. Jeffy was therefore run against widely used open-source projects with no connection to its author, using a local clone, with nothing going upstream without a filed issue or pull request. Every run is published in full: the journal, the report, the patch where the run changed code, and the link to the upstream issue or pull request where one was filed [25].

Projects tested	Converged	Failed	Pull requests merged	Pull requests open	Issues filed
129	100	28	20	36	4

One hundred projects ran to convergence across thirteen languages: Python 19, Rust 14, Go 12, JavaScript 10, C++ 7, Ruby 6, C 5, Java 5, PHP 5, Swift 5, TypeScript 5, Kotlin 4, C# 3. Twenty-eight never converged inside the budget they were given, and every one is published with that budget and the reason it ran out.

The maintainers are the judges

Twenty pull requests the loop wrote are merged across seventeen projects, because a maintainer with no stake in this project reviewed each one and said yes. Only fixes the loop had scored High, reproduced on the base commit and proven green on the fix went upstream, one pull request per finding, verified in a fresh clone before filing. The verdicts, verbatim where the maintainer left one:

- **mimalloc (Microsoft)**, C, 3 runs, 28 iterations. The zeroing allocator returned uninitialized memory above the small-size threshold. The author's whole review: "Yikes - - terrible. Thanks for the PR." Merged eight hours after filing.
- **ada (the URL parser inside Node.js)**, C++, 2 runs, 18 iterations. A host boundary was reported one byte short of its documented position. Merged twelve minutes after filing.
- **kotlinx-datetime (JetBrains)**, Kotlin, 2 runs, 17 iterations. The IDE quick-fix on a deprecated method swapped epoch milliseconds for nanoseconds and left no warning behind. Merged 88 minutes after filing.

- **swift-log (Apple)**, Swift, 2 runs, 16 iterations. A documented no-op setter asserted instead. The maintainer wanted the assertion kept; the merged change is the documentation saying so. Merged the next morning.
- **commons-text (Apache)**, Java, 3 runs, 24 iterations. An overload forwarded the buffer end as its start, so custom matchers never matched. The reviewer opened with "Nice find, thank you" and asked for two more tests; delivered in 24 minutes, merged 36 minutes after that.
- **urfave/cli**, Go, 2 runs, 18 iterations. A quoted argument with leading whitespace reached the action stripped. The reviewer asked about four flag-like inputs; answered with a measured table; merged.
- **console**, Rust, 1 run, 12 iterations. A truncation sliced a string at a column count used as a byte offset and panicked mid-character. Merged after one review round.
- **indicatif**, Rust, 2 runs, 20 iterations. Width arithmetic underflowed when the last drawn line was wider than the terminal. Merged after one review round.
- **nanostores**, JavaScript, 2 runs, 18 iterations. A batch inside a listener replayed listeners, and an unbind detached the wrong listener on its second call. Two pull requests, both merged for the next minor release.
- **go-runewidth**, Go, 2 runs, 20 iterations. A wrap at width zero divided by zero, and a lookup table never rebuilt after a flag change. Two pull requests, both merged within an hour of filing.
- **unicode-segmentation**, Rust, 2 runs, 15 iterations. A size hint on the empty string violated the iterator contract. Merged.
- **nanoid**, JavaScript, 2 runs, 16 iterations. An alphabet the generator could not sample from hung forever. The maintainer judged a runtime guard too expensive and asked for a documentation notice; reworked and merged the same day.
- **csv (thehpleague)**, PHP, 4 runs, 37 iterations. A separator was matched as a regular expression, and an empty field was escaped as a formula. Two merged; a third was closed after the maintainer explained the design and the author conceded.
- **bat**, Rust, 1 run, 10 iterations. A just-merged security flag did nothing when piped; caught before it ever shipped.
- **fasthttp**, Go, 7 runs, 58 iterations. A Content-Length no parser should accept became a wrong number.
- **jsoncpp**, C++, 1 run, 10 iterations. The documented secure build had never compiled on one compiler, and the memory it promised to wipe was only partly wiped.
- **PapaParse**, JavaScript, an audit rather than a loop run. Header de-duplication re-ran on every row after a resume and rewrote data in place. Merged and shipped in 5.7.0.

One more finding was fixed upstream by the maintainer himself: in `chalk`, the project owner reproduced the loop's finding, wrote his own fix, and shipped it in the next major release. And one finding is not counted as a fix at all: a security finding the loop produced in a repository of

Anthropic's is open with Anthropic's own security program, scored Low, with their review ongoing and the details withheld at their request until it resolves.

The rejections taught the most, and they are published beside the merges. An Apple engineer closed a pull request by pointing at a design decision he had recorded a year earlier in an issue the loop's search had not found; the pull request was conceded and closed the same day, and the duplicate check now reads the closed issues on the function a change touches, not the titles that match its keywords. Another Apple engineer wanted an assertion kept where the loop had removed one; the bug was real, his reasoning about the contract was better, and the doc-only version he asked for is the one that merged. Two maintainers in two days used the same word for the pull request prose, and the rule since then is three short paragraphs and no new comments in someone else's code. One pull request was closed as a duplicate of a pull request that was already open, because the search that was supposed to catch it could not tell a pull request from an issue. Each of those is a lesson written at mechanism level, and each is in the repository rather than in anyone's memory.

The failures, and what they taught about shape

Twenty-eight projects did not converge, and the ledger of every attempt holds thirty-four that ran out of budget, counting the first attempts of projects that converged only on a retry. They are the more instructive half of the corpus. From the fifth wave onward every target carried the same pre-registered budget, five rounds of ten iterations, declared in a file before the first iteration ran, so that a convergence could not come from extending a budget after watching it fail. The failures then have a shape, and it is not size and it is not popularity. `mruby`, a Ruby interpreter in C, ran ten rounds and 113 iterations and its evaluator gate was never once due: on a surface that large the ledger never emptied with a clean audit standing. `eemeli/yaml`, a mainstream JavaScript library of ordinary size, showed the same shape in forty iterations. `BurntSushi/toml` was rejected seven times across five runs, every rejection the same class, a claim in a state file asserting more than the check behind it drove. `picomatch`, a glob compiler with two independent parse paths, swept its whole surface in the first round and then spent four rounds blocked on decisions only its owner could make, which the loop had no way to know its owner had already made in a published advisory. What converges is the single-purpose library with a fast oracle: a date library, an allocator, a URL parser, a CSV reader. What mostly does not is the specification implementation, the parser with two paths, the interpreter, whatever its star count. A 392-star Apache library with five outside contributors in the last four months converged in two rounds; star counts predicted nothing.

That is also why this paper states no convergence rate. One hundred of 129 is not a rate. The targets were chosen for properties, an external oracle, an active upstream, a healthy suite, so the denominator is not a sample of anything, and the early targets ran until they converged, which is outcome-conditional stopping and cannot produce one. What can be quoted is the cost, which is both true and harder to attack: the receipts say what every convergence took in rounds and iterations, and every non-convergence carries the budget it was given before it started.

Two entries are worth as much as the dramatic ones, for the opposite reason. The `RuboCop` run swept every department of a mature Ruby linter, individually re-proved the last twenty upstream

commits, filed nothing, and changed not a single line. The `gson` run took two iterations, found one Low-severity issue, priced it, declined it, and stopped. A method that always finds something is not measuring the code; it is measuring its own eagerness. The null results are the control group, and they are published in the same table as the successes.

The clearest single finding is still the shortest. In `kennethreitz/records`, a Python database library, `pytest` reported 31 tests passing. At that same commit, four separate High-severity defects were live, all of them data loss: an `INSERT` committed nothing and the rows vanished silently on close, bulk inserts did the same, a bare exception handler swallowed every transaction failure so callers believed failed writes had succeeded, and every query leaked a pooled connection. The suite stayed green through all four because its fixture used an in-memory database sharing a single connection, where uncommitted state is always visible. The gate was measuring the wrong world. That is the failure mode this entire system exists to catch.

The greenfield arm: three builds judged by suites the loop did not write

Every receipt above is brownfield: an existing project arrived with an existing test suite, so the gate predated the loop. Section 17 records the residual weakness even so - the loop writes many of the tests that certify the loop. Greenfield is that weakness at its maximum. With no pre-existing code there is no pre-existing oracle, and a naive greenfield run would author one hundred percent of its own gate and converge against homework it set itself. The pre-registered answer is to hand the judgement to a suite the loop cannot edit. Three targets were built. The first two were chosen to be different oracle classes, because that difference is the argument rather than an incidental detail; the third answers the objection both of them invite, and is described after them:

	Corpus author	Judge	Code author
Brownfield (the table above)	external	external	external
Target 1, TOML decoder	external	external	the loop
Target 2, gitignore matcher	the loop	external	the loop
Target 3, TOML-M decoder	external	external, through a frozen adapter	the loop

For each target, `PLAN.md` was committed before iteration 1 - the goal, the operating envelope, the surface inventory, and a `Verify` command naming the external judge - so `git log` proves the gate predates the work. One deliberate deviation from this project's own documentation is recorded here rather than discovered: `BACKLOG.md` shipped empty, against the Scoped-mode advice to seed finite tasks, so that the task decomposition would be the loop's work and not the author's. The engine was not modified for any target, and no run was intervened in.

Target 1 is a TOML v1.0.0 decoder in Rust, judged by `toml-test` v2.2.0 [26]: 205 valid cases, 474 invalid, 679 assertions. The inventory is external too - the suite's own test groups supply the 17 surface-inventory rows, which makes this target stricter than any brownfield target on that axis, since in brownfield the loop writes the inventory from the code it reads. The zero measurement before iteration 1 was the suite failing on the first case because the binary did not exist. Eleven

iterations later, in a single run, the suite reported every assertion passing and the adversarial evaluator passed the declaration on its first invocation, after twenty hostile probes of its own. One rule from the pre-registration matters for reading the numbers: valid and invalid counts are always reported separately, never as one fraction, because a stub that rejects every input scores 474 of 679 assertions while decoding nothing - a single collapsed pass rate is gameable by refusal.

Target 2 is a `.gitignore` matcher in Rust with no packaged suite in existence, so the judge is the real `git` binary [27]: a differential harness materializes a repository per case in an isolated configuration and compares the matcher's verdict against `git check-ignore -v` on every query, with an oracle error never readable as agreement. The 12 inventory rows were frozen from `gitignore(5)` before iteration 1. The corpus, though, is the run's own - that is the deliberately weaker oracle class, and section 17 weighs it - mitigated by freezing: the corpus only ever grew, 53 cases at the freeze to 106 at convergence, and its size is printed beside every pass count. The final position is 106 cases, 300 queries, zero disagreements against `git 2.50.1` on Windows, and the verdicts are environment-specific by construction: the oracle ran on NTFS with `core.ignoreCase true`, which is how the run came to implement `git`'s actual casefold semantics rather than the manpage's.

Target 3 is that answer, pre-registered before either of the first two was written up: both formats saturate any plausible training set, so convergence there may measure recall rather than method. TOML-M is TOML v1.0.0 with two rules deliberately inverted - `true` and `false` denote the opposite values, and `tab` and `line feed` swap inside string values, key names excepted - so recalling the real format produces wrong answers. Because both amendments are involutions, the unmodified `toml-test` binary remains the judge through a frozen sixty-one-line adapter that applies the same swap to the decoder's output before comparison; the adapter never reads the input, never edits an expectation, and is byte-identical at the converged commit. The receipt reports two numbers from one binary: 205 of 205 against the mutated suite, and 169 of 205 against standard TOML - failing on precisely the cases the amendments touch, which is the mirror image of the penalty measured on target 1's decoder before this run began. It built the dialect rather than recalling the format, at a cost of fourteen iterations against eleven for the unmutated build.

Target	Spec	Judge	Final position	Rows swept	Iterations	Runs
TOML decoder, Rust	TOML v1.0.0	<code>toml-test v2.2.0</code>	205 of 205 valid, 474 of 474 invalid	17 of 17	11	1
<code>gitignore</code> matcher, Rust	<code>gitignore(5)</code>	<code>git check-ignore 2.50.1</code>	106 cases, 300 queries, 0 disagreements	12 of 12	42	5
TOML-M decoder, Rust	TOML v1.0.0, two rules inverted	<code>toml-test v2.2.0</code> via a frozen adapter	205 of 205 valid, 474 of 474 invalid; 169 of 205 against standard TOML	17 of 17	14	1

Target 3's receipt also carries a disclosed process violation, and it is recorded here rather than left for a reader to find. The engine grants one two-iteration extension at budget exhaustion and states that the window buys only the closing ceremony - never an audit. At budget end this run had no clean audit on record: iteration 11 filed a defect, iteration 12 fixed it, and the run's own journal states that convergence should fall to the next run. Iteration 13 then ran a full audit inside that window, and iteration 14 declared convergence citing it. The result's substance is unaffected and re-derivable from a clone; what is affected is the process claim. The reason was instructive: the rule existed only as prose in the iteration prompt, and the shell gate that enforces every other convergence condition could not see an audit entry inside the window. That gap is closed. An audit entry at an iteration inside the closing window now ends the run out of budget before any declaration is read. The disclosure stands as written, because the run happened under the engine that permitted it, and the engine changing afterwards is the outcome the disclosure was for.

The gitignore arc is the part a skeptic should read first, because it is a record of the gate refusing. The frozen corpus was fully green from the first iteration of the second run; everything after that - nearly four runs of it - was the adversarial evaluator probing beyond the corpus and declining to countersign. It was invoked eight times across runs 2 through 5 and rejected seven, each rejection substantiated by disagreements reproduced through the run's own harness, and its thirteen filed findings marched steadily outward from matcher semantics into territory that is genuinely obscure: `wildmatch.c`'s escaped-slash clause, git's own `isspace` accepting exactly four bytes, NTFS folding names the matcher compared byte-wise, 8.3 short-name aliases whose byte length differs from the directory they name, and the layer boundary where Win32 normalization strips trailing dots that git's extended-length file opens never see. Three runs ended blocked, out of evaluator invocations, and each block is published as the receipt it is. Those blocks are also why the rule changed: two of them died at a second rejection that was still holding an invocation, sitting on six budgeted iterations each, and the terminal-rejection rule and the gate-salvage path described in section 14 are the engine's answer to that record. Once, the loop refuted its own gate: the fourth rejection arrived with three reasons, the loop reproduced two and disproved the third with direct oracle evidence, filed the two and recorded the refutation. The state files - journal, plan, backlog - ship inside both repositories [26][27], which reverses this project's own rule of keeping its development journal out of the published tree: for a brownfield fix the upstream diff is the evidence, and for a greenfield build the process is.

The claim, stated narrowly: the same engine, unmodified, in Scoped mode, converged on a build from an empty directory whose completeness was decided by a judge it did not write - including one specification where recalling the real format actively produces failures. Four things are explicitly not claimed. Not a completion rate - three chosen targets are not a sample. Not an answer to Huntley's ninety percent, which is a self-assessed rate over many projects. Not a statement about delivery quality - conformance says the decoder matches the spec, nothing about whether anyone would want the API. And not invention, for the reason section 17 states.

The loop improves the loop

The rule that governs every other project also governs this one: Jeffy is pointed at its own repository exactly as it is pointed at a stranger's, same envelope, same evidence rules, same adversarial gate, and the results are merged in public. When a run finds a defect in the engine, the fix is not a warning in the documentation but a behavioural check in the validator that fails if the defect returns, which is how the suite reached 318. The current release is the freshest example. The day it shipped, the engine was run against itself, off the record, and inside an hour it filed two Mediums against the release: a command in the new launch pre-flight whose delimiter collided with the text it was parsing, so that every launch would have reported itself as the first run of every project; and the severity ceiling described above, which the release had put in the template a new project copies and nowhere a project adopted before the rule could read it. The loop moved the rule into the iteration prompt, which every project reads at every turn, wrote the validator check that keeps the prompt and the hook from drifting apart again, and fixed the command. Both are in the release that follows. That is the mechanism working as designed, and it is the reason a paper about this engine will always be a little behind it.

17. Honest limits

Every result above has an edge, and the edges are where the next release comes from. They are stated here as plainly as the results.

The verifier shares the generator's blind spots. The adversarial evaluator is the same model with fresh context. It defeats self-persuasion; it does not defeat a defect in the training data. There is a documented case. In the python-dotenv run, the loop filed a finding claiming that variable-default syntax contradicted the library's documented precedence. The evaluator passed it. Only a later verification, performed by agents working from a clean clone that had never seen the loop's tree, established that the premise was false. The finding is retracted in the published receipt rather than quietly dropped.

The loop writes many of the tests that certify the loop. In that same project the suite grew from 220 tests to 511, and 291 of those are the loop's own. They are falsifiable, and restoring the original source files fails exactly the tests that pin the run's fixes. But they were written by the same process that wrote the fixes, and a reader should discount them accordingly. The greenfield arm in section 16 is the pre-registered reply to this limit, and the merged pull requests are the stronger one: a maintainer merging a fix into their own library has discounted the loop's tests already and decided the change stands on its own.

A green gate is only as wide as the command behind it, and the engine cannot see how wide that is. The go-yaml run converged with a verify command that exited zero at every gate and never once executed the 402-case conformance corpus sitting in the repository it was auditing. The oracle declaration forces a run to name what its command grades and what its platform excludes, which converts a silent gap into a written claim a reader can check. That is a real improvement and not a solution. A shell script still cannot judge whether a declared oracle is the right one or a wide enough one. The general form of the limit survives every mechanical fix

proposed for it: the loop can only be as honest as its oracle is real, and whether an oracle is real is a judgement about the project.

A frozen self-authored corpus is not an external corpus. The gitignore target's judge is git itself, but the questions put to that judge were generated by the run, so the run controlled the denominator. The mitigations are real - the corpus was frozen before any matching code existed, never shrank across five runs, and its size is printed beside every pass count. Two further checks were run after convergence rather than claimed at it, and both are reproducible from the published repository [27]: the 53 cases frozen before the matcher existed replay alone at zero disagreements, and a cold corpus of 40 cases authored blind by a fresh context that never read the source replays at zero disagreements too. That is a stronger position than freezing alone, and it is still not an external corpus, because the blind author is the same model that wrote the matcher.

A model that has read many TOML parsers is not building from nothing. Target 3 was pre-registered to test exactly that objection, and the result shows the loop built to the specification rather than reciting the format. That is evidence about stopping discipline under an external judge, not about invention. A genuinely novel specification with no public implementations remains a harder test, and it has not been run.

A known-answer check cannot argue about which answer the context demands. In `quantstats`, a financial statistics library, a later manual audit found six defects the loop did not, all still present at the converged tree, and all convention defects: internally consistent numbers that are wrong in context. A bet-sizing function implemented the textbook formula faithfully, and the textbook formula produces a result unchanged when every return is halved, which no capital-allocation recommendation can be. The loop's probes validated the same formula the code implemented, certifying the convention instead of testing it. A known-answer probe can verify that a formula was computed correctly. It cannot adjudicate which formula the situation calls for. That requires a domain expert.

Convergence is a statement about a contract, not a certificate of correctness. It says that under a declared envelope and sweep contract, one full fresh-evidence audit found nothing High or Medium on a fully swept surface, an adversarial reviewer failed to break that result, and a shell script re-verified the account. It does not say there is nothing left, and it never will.

Platform monoculture hides defects the same way test monoculture does. Every converged run executed on Windows or Linux. When a macOS continuous integration leg was added to the engine's own suite, it went red on its first run and named a real defect: the verify gate probed for a command-line utility before running any of its checks, and stock macOS does not ship that utility, so on every Mac the gate skipped itself silently. It went red again three weeks later on a `sed` dialect that only BSD reads differently. Repetition is not independence, whether the thing repeated is a model, a test fixture, or an operating system.

The loop's own bookkeeping cost more than its code did. Read across the published non-convergences, the dominant rejection class was never a bad fix. It was a claim the run wrote into its own state files that the gate re-derived and found false: a row swept at a commit its code had moved past, a count typed beside the command that contradicts it, a declined finding whose

premise was not true. The rules in section 14 that make numbers executable and rows derivable are the answer, and the lint mode is the latest instalment, and a reader should expect the next release to be another one. An engine whose stopping decision is a script re-reading the account will spend its improvement budget on the account. That is the cost of the thesis, and it is a cost worth paying.

The strongest available check is the one with no stake in the result. This is why twenty merged pull requests across seventeen projects carry more evidential weight than one hundred convergence declarations. A maintainer reviewing a patch against their own library shares none of the loop's priors, none of its context, and none of its incentives. Every other check in this document is, ultimately, the system grading itself with progressively better discipline. Only the maintainer is genuinely outside it, and seventeen of them said yes.

Glossary

Terms are defined here in the order a newcomer is likely to meet them.

Large language model. A system trained to predict text, which in practice can follow instructions, write code and explain its reasoning. On its own it produces text and nothing else.

Context window. The finite amount of text a model can consider at once, holding the conversation, the instructions, and any file contents or tool output loaded into it. It is the scarcest resource in agent design, and quality degrades as it fills.

Tool call. A structured request from the model for something to happen outside itself, such as reading a file, running a command or searching the web. The result is returned to the model as text.

Agent. A model placed in a loop with tools, so that it can act, observe the result, and act again without a human between each step.

Harness. The surrounding program that runs the agent: it holds the loop, supplies the tools, manages the context, and decides when to stop.

Workflow. A system where models and tools are orchestrated along predefined code paths. Distinguished from an agent, where the model itself decides what to do next.

Oracle. Anything outside the model that can settle whether an answer is right. A test suite, a compiler, a mathematical identity, a specification, or a human expert. Coding is a favourable domain for agents mainly because it has cheap oracles.

Ground truth. Information about the real state of the world, obtained from the environment rather than generated by the model. A passing test is ground truth; the model's belief that the code works is not.

Hook. A script the harness runs at a fixed point in the agent's lifecycle. Unlike written instructions, which the model may or may not follow, a hook always executes.

Stop hook. A hook that runs when the agent tries to finish its turn, and which may refuse to let it finish. This is what turns a single-shot session into a loop.

Subagent. A second agent instance with its own separate context window, given a narrow task and returning only its conclusion. Used to explore without polluting the main context, or to review work without having seen the reasoning that produced it.

Evaluator. A model instance whose job is to judge another's output rather than produce work of its own.

Adversarial evaluator. An evaluator instructed to assume the work is broken and to try to break it, rather than to assess it neutrally.

Iteration. One pass through the loop: one unit of work, its verification, and its record.

Operating envelope. A written statement of what a piece of software is expected to receive, from whom, and under what assumptions. Severity of a defect is judged relative to it.

Surface inventory. A checklist of every public part of a project, used to ensure an audit covers the whole surface before investigating any part of it deeply, and to make clear which parts were examined.

Acceptance check. A specific command, recorded with a task, that determines whether that task is genuinely complete.

Verify command. The project's own overall gate, usually its test suite, run after every change to confirm nothing has broken.

Checkpoint. A commit made automatically at the end of each iteration, serving as the point the loop reverts to if the next change breaks the build.

Convergence. The loop's term for a finished run: a state in which a fresh audit found nothing significant on a fully examined surface, no work remains open, the project's tests pass, an adversarial reviewer countersigned, and a shell script independently re-verified all of it.

Receipt. A published record of one run against a real project, containing the journal, the resulting patch, and any upstream issue or pull request, so the claims can be checked by a reader.

Severity floor. The rule that a run may declare convergence with Low findings still open, provided each is carried by name into the declaring entry and the receipt. Only a High or a Medium blocks.

Severity ceiling. The rule that a High or a Medium must name a consequence a user of the shipped product meets. A finding about tests or developer tooling is a Low, always.

Probe battery. A set of known-answer checks the loop writes for one part of a project's surface, kept under `.jeffy/probes/` and committed, with a claims file listing every measurement it makes as a command the engine can re-run.

Claims file. The list of measurements a probe battery makes, one per line as `expect <value> :: <command>`. A number the run states must be one of these, or a row in the equivalent table for the plan file, so it is re-derived rather than believed.

Closing window. A one-time two-iteration extension the hook grants when the budget expires with the work done, which buys only the evaluator gate, the fixes it files, and the declaration, never another audit.

Ratchet. The rule that a run relaunched on a tree that has not changed since an accepted convergence re-verifies that and stops at once rather than auditing the same code again.

Lint mode. The hook run read-only, on demand, to print what a declaration would be refused on. The loop runs it before every evaluator invocation so that a bookkeeping refusal never costs a review.

References

All sources accessed 4 August 2026. Where a publication date is contested or absent, that is stated rather than resolved silently.

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. "ReAct: Synergizing Reasoning and Acting in Language Models." arXiv:2210.03629, 6 October 2022. Published at ICLR 2023. <https://arxiv.org/abs/2210.03629>
2. Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. "Reflexion: Language Agents with Verbal Reinforcement Learning." arXiv:2303.11366, 20 March 2023. <https://arxiv.org/abs/2303.11366>
3. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., and others. "Self-Refine: Iterative Refinement with Self-Feedback." arXiv:2303.17651, 30 March 2023. <https://arxiv.org/abs/2303.17651>
4. Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., and Zhou, D. "Large Language Models Cannot Self-Correct Reasoning Yet." arXiv:2310.01798, 3 October 2023. Published at ICLR 2024. <https://arxiv.org/abs/2310.01798>
5. Valmeekam, K., Marquez, M., and Kambhampati, S. "Can Large Language Models Really Improve by Self-critiquing Their Own Plans?" arXiv:2310.08118, 12 October 2023. <https://arxiv.org/abs/2310.08118>
6. Richards, T. B., and contributors. AutoGPT, Significant Gravitas. Repository created 16 March 2023; first tagged release 12 April 2023. <https://github.com/Significant-Gravitas/AutoGPT>. The README warning on continuous mode and the issue reports cited in section 3 are in that repository, notably issues 920 and 3644, both closed as not planned. The widely cited release date of 30 March 2023 has no corresponding repository artefact.
7. Nakajima, Y. "Task-driven Autonomous Agent Utilizing GPT-4, Pinecone, and LangChain for Diverse Applications." 28 March 2023. <https://yoheinakajima.com/task-driven-autonomous-agent-utilizing-gpt-4-pinecone-and-langchain-for-diverse-applications/>. Code published as BabyAGI, repository created 3 April 2023. <https://github.com/yoheinakajima/babyagi>
8. Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., and Scialom, T. "GAIA: a benchmark for General AI Assistants." arXiv:2311.12983, 21 November 2023. <https://arxiv.org/abs/2311.12983>
9. Liu, X., and others. "AgentBench: Evaluating LLMs as Agents." arXiv:2308.03688, 7 August 2023. Published at ICLR 2024. <https://arxiv.org/abs/2308.03688>
10. Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" arXiv:2310.06770, 10 October 2023. Published at ICLR 2024. <https://arxiv.org/abs/2310.06770>
11. Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering." arXiv:2405.15793, 6 May 2024. <https://arxiv.org/abs/2405.15793>
12. Gauthier, P. Aider. Repository created 9 May 2023. <https://github.com/Aider-AI/aider>. Documentation on linting and testing at <https://aider.chat/docs/usage/lint-test.html> and on git integration at <https://aider.chat/docs/git.html>
13. OpenAI. "Why we no longer evaluate SWE-bench Verified." 23 February 2026. <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>
14. Huntley, G. "Ralph Wiggum as a 'software engineer'." 14 July 2025. <https://ghuntley.com/ralph/>. Secondary sources place the technique's origin earlier in 2025, variously in February, May and June; only the publication date of this post is independently verifiable.
15. Farr, C. ralph-playbook. Repository created 9 January 2026. <https://github.com/ClaytonFarr/ralph-playbook>. Frequently miscited as Huntley's own work; the repository under his account is a fork of this one.

16. Huntley, G. "everything is a ralph loop." 17 January 2026. <https://ghuntley.com/loop/>
17. Anthropic. ralph-loop plugin, claude-plugins-official. Originally named ralph-wiggum and renamed 6 January 2026. <https://github.com/anthropics/claude-plugins-official> . The termination behaviour cited in section 6 was read from the plugin's Stop hook implementation.
18. OpenAI. "Addendum to o3 and o4-mini system card: Codex." 16 May 2025. Section 2.3, "Falsely claiming to have completed a task that it did not complete." https://cdn.openai.com/pdf/8df7697b-c1b2-4222-be00-1fd3298f351d/codex_system_card.pdf
19. Advani, L. "From Confident Closing to Silent Failure: Characterizing False Success in LLM Agents." arXiv:2606.09863, 1 June 2026. <https://arxiv.org/abs/2606.09863>
20. Anthropic. "Building effective agents." 19 December 2024. <https://www.anthropic.com/engineering/building-effective-agents>
21. Anthropic. "Best practices for Claude Code." Living documentation carrying no publication date; accessed 4 August 2026. <https://code.claude.com/docs/en/best-practices>
22. Zhang, B., Lazuka, K., and Murag, M. "Equipping agents for the real world with Agent Skills." Anthropic, 16 October 2025. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
23. Anthropic Applied AI team. "Effective context engineering for AI agents." 29 September 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
24. Rajasekaran, P. "Harness design for long-running application development." Anthropic, 24 March 2026. <https://www.anthropic.com/engineering/harness-design-long-running-apps>
25. Jeffy Loop. Source, receipts and validator. <https://github.com/lenamonj/jeffy-loop> . Every figure in Part III was recomputed from that repository at the commit current on 5 September 2026, release v1.21.0 (b79d32f); the counts it states are the markers the repository's own validator derives from the receipts table at that commit.
26. Jeffy Loop greenfield receipt: TOML 1.0 decoder, judged by tom1-test v2.2.0. <https://github.com/lenamonj/jeffy-greenfield-toml> . The pre-registration, journal, backlog and every iteration commit ship in the repository; the final suite output was re-derived from a fresh clone at the converged commit.
27. Jeffy Loop greenfield receipt: gitignore matcher, judged differentially against git check-ignore. <https://github.com/lenamonj/jeffy-greenfield-gitignore> . The pre-registration, the frozen corpus, and the journals of all five runs, including the three that ended blocked, ship in the repository; the final differential output was re-derived from a fresh clone at the converged commit.
28. Jeffy Loop greenfield receipt: TOML-M, a deliberately mutated dialect judged by the unmodified tom1-test suite through a frozen output adapter. <https://github.com/lenamonj/jeffy-greenfield-toml-mutated> . The pre-registration, the adapter, and the full journal ship in the repository, as does the disclosed process violation described in section 17; both reported suite results were re-derived from a fresh clone at the converged commit.

The source, every receipt cited in Part III, and the validator that holds the engine to its behavioural check suite:

<https://github.com/lenamonj/jeffy-loop>